BONUS žingsnis (+2 balai):

Įvadas į Kopijos konstruktorių (Copy constructor)

Direktorija: `U3/BONUS/`

Motyvacija:

U3 3 žingsnyje **sąmoningai užkomentavome** `Window` ir `Door` konstruktorių ir destruktorių [DEBUG] logging'ą ("elgsenos protokolą").

Kodėl? Jei jį įjungtume, pamatytume **keistą** dalyką `Window` objektų gyvavimo cikle:

> Konstruktorių: 2
> Destruktorių: 5
> Counter pabaigoje: **-3 ?**

Atėjo metas pasiaiškinti — ką dar **papildomai** daro kompiliatorius kurdamas objektus. Konkrečiau šios užduoties kontekste: Kokius veiksmus "C++ sistema" atlieka "užkulisiuose", kad parūpintų į `Room` įkomponuoto `Window` vektoriaus valdymą.

Pastaba: `vector<Objektas>`, kaip duomenų/objektų konteinerį/talpyklą, pasirinkome ir naudojome jau ankstesnėse užduotyse, kad nuo pat pradžių, šalia bazinių, C paveldo priemonių ("*kreivų*" masyvų, besiremiančių rodyklių aritmetika), "pajusti ir pačiupinėti" modernias, patogesnes, "natūralesnes" C++ priemones.

Šio žingsnio eiga:

1. Nukopijuokite `04/` kodą į `BONUS/`

2. Atkomentuokite logging kode:

Window.cpp:

```
Window::Window(float height, float width) {
    ...
    std::cout << "[DEBUG] Window created..." << std::endl;  // ← Atkomentuoti!
}

Window::~Window() {
    ...
    std::cout << "[DEBUG] Window destroyed..." << std::endl;  // ← Atkomentuoti!
}
```

Door.cpp — taip pat.

3. Kompiliuokite ir paleiskite Žingsnis 4 programą:

Tikėtina išvestis:

```
=== Kambario skaičiavimai ===
[DEBUG] Room created (5×4×3)

--- Pridedame langus ---
[DEBUG] Window created (1.5×1). Total: 1
[DEBUG] Window created (1.5×1). Total: 2
Window 1 area: 1.5 m²
Window 2 area: 1.5 m²
[INFO] Window added. Total: 1
[DEBUG] Window destroyed. Remaining: 1      ← ❓ Kodėl iškart?!
[INFO] Window added. Total: 2

--- Pridedame duris ---
[DEBUG] Door created (2×0.9). Total: 1
Door area: 1.8 m²

--- Kambario informacija ---
Room dimensions: 5×4×3 m
Windows: 2
Door: 2×0.9 m

--- Skaičiavimai (globalios funkcijos) ---
Wall area (minus windows/door): 49.2 m²
Baseboard length (minus door): 17.1 m

--- Atlaisviname atmintį ---
[DEBUG] Door destroyed. Remaining: 0

--- Programa baigiasi ---
[DEBUG] Window destroyed. Remaining: 0
[DEBUG] Window destroyed. Remaining: -1    ← ❓ Pradėjo "dūžti" nesami langai?
[DEBUG] Room destroyed
[DEBUG] Window destroyed. Remaining: -2    ← ❓
[DEBUG] Window destroyed. Remaining: -3    ← ❓
```

Sprendimas:

Pasirodo, yra ir kitas konstruktorius - vadinamasis **Kopijos konstruktorius** (Copy Constructor)

Kopijos konstruktorius (KK) — specialus konstruktorius, kuris sukuria objektą **kopijuodamas** (galime sakyti ir - klonuodamas) kitą objektą.

Sintaksė (KK prototipas):

```
ClassName(const ClassName& other);
```

Žinome, kada kviečiami konstruktoriai ir destruktoriai.

O kada kviečiamas KK?:

```
Window w1(1.5, 1.0);      // kviečiamas (parametrinis) konstruktorius
Window w2 = w1;           // kviečiamas KK
Window w3(w1);            // kviečiamas KK
_vector_.push_back(w1);   // KK kviečiamas ir didinant vektorių!
```

O jei jo nėra?

Tuomet kompiliatorius jį **generuoja automatiškai** (implicitly define).

Prisiminkime: kompiliatorius taip pat elgiasi ir generuodamas kitus specialiuosius metodus: konstruktorių ir destruktorių.

```
// Kompiliatorius sugeneruoja, pvz.:
Window(const Window& other)
    : height(other.height), width(other.width)
{
}
```

```
// arba nenaudojant narių inicializavimo sintaksės:
Window(const Window& other) {
    this->height = other.height;
    this->width = other.width;
}
```

🎯 Šio žingsnio tikslas:

> Pasitelkus kopijos konstruktorių logging'ą išsamiau pademonstruoti objektų gyvavimo ciklą

Reikalavimai naujam kodui:

- **Pridėkite Kopijos konstruktorius į `Window` ir `Door` klases:**
- **Kopijos konstruktoriuose:**

```
ClassName(const ClassName& other) {
    // UŽDUOTIS 1: Nukopijuoti member kintamuosius
    // UŽDUOTIS 2: Padidinti static skaitiklį
    // UŽDUOTIS 3: Logging'ą atlikti naudojant
    // std::cout << "[DEBUG] ClassName COPIED ...
}
```

Testavimas:

main.cpp: (tas pats kaip 04/)

Tikėtina išvestis (su KK):

```
=== Kambario skaičiavimai ===
[DEBUG] Room created (5x4x3)

--- Pridedame langus ---
[DEBUG] Window created (1.5x1). Total: 1
[DEBUG] Window created (1.5x1). Total: 2
Window 1 area: 1.5 m²
Window 2 area: 1.5 m²
[DEBUG] Window COPIED (1.5×1). Total: 3
[INFO] Window added. Total: 1
[DEBUG] Window COPIED (1.5×1). Total: 4
[DEBUG] Window COPIED (1.5×1). Total: 5
[DEBUG] Window destroyed. Remaining: 4
[INFO] Window added. Total: 2

--- Pridedame duris ---
[DEBUG] Door created (2x0.9). Total: 1
Door area: 1.8 m²

--- Kambario informacija ---
Room dimensions: 5×4×3 m
Windows: 2
Door: 2×0.9 m

--- Skaičiavimai (globalios funkcijos) ---
Wall area (minus windows/door): 49.2 m²
Baseboard length (minus door): 17.1 m

--- Atlaisviname atmintį ---
[DEBUG] Door destroyed. Remaining: 0

--- Programa baigiasi ---
[DEBUG] Window destroyed. Remaining: 3
[DEBUG] Window destroyed. Remaining: 2
[DEBUG] Room destroyed
[DEBUG] Window destroyed. Remaining: 1
[DEBUG] Window destroyed. Remaining: 0
```

Failų struktūra:

```
U3/BONUS/
├── Window.h           ← Su KK deklaracija
```

```
├── Window.cpp         ← Su KK implementacija
├── Door.h             ← Su KK deklaracija
├── Door.cpp           ← Su KK implementacija
├── Room.h, Room.cpp   ← kaip 04
├── calculations.h, calculations.cpp  ← kaip 04
├── main.cpp           ← pradžioje kaip 04
└── Makefile
```

Papildomai:

Eksperimentuokite su skirtingu langų skaičiumi:

```
Window w2(2.5, 1.0);  // Skirtingi matmenys
Window w3(3.5, 1.0);  // 3 langai
Window w4(4.5, 1.0);  // 4 langai
```

Stebėkite kaip kinta konstruktorių, destruktorių ir kopijos konstruktorių skaičius. Pabandykite paaiškinti programos elgseną.

... bus papildyta aiškinamąja/metodine medžiaga